

中山大学

《软件工程》期末大作业

软件测试与质量保证报告

项目名称：逸仙活动云——中山大学校园活动云平台

小组成员：钟梓俊 邱剑波 张宸睿

指导老师：王可泽

2026 年 7 月 16 日

目录

1	测试目标	1
2	测试范围	1
2.1	功能范围	1
2.2	非功能范围	1
2.3	排除范围	2
3	测试策略	2
3.1	单元测试	2
3.2	集成测试	2
3.3	系统测试	2
3.4	验收测试	3
4	测试过程	3
4.1	测试环境	3
4.2	测试计划	3
4.3	测试用例设计	4
4.4	测试执行结果	6
5	缺陷管理	7
5.1	缺陷分类与严重度	7
5.2	缺陷生命周期与字段	7
6	质量保证方法	8
6.1	代码审查	8
6.2	静态检查	9
6.3	自动化测试	9
6.4	回归测试	9
6.5	发布检查	10
7	测试结论	10

1 测试目标

测试的目标是验证逸仙活动云系统在正常、边界和异常条件下满足需求分析中定义的功能和非功能需求，并通过自动化回归降低迭代过程中的缺陷逃逸风险。具体包括：

1. 验证核心业务功能（认证、活动发现、发布审核、个人参与）的正确性和可用性。
2. 验证系统在异常输入、网络故障和越权访问下的防护能力和错误处理正确性。
3. 验证前后端接口契约的一致性，即前端期望的数据结构、状态码和错误语义与后端实现保持一致。
4. 通过自动化测试建立回归防护网，确保功能迭代不破坏已有正确行为。

2 测试范围

2.1 功能范围

本次测试覆盖需求分析中定义的八项功能域：

1. **活动发现与检索（FR-01）**：活动列表展示、关键词搜索、分类/状态筛选、分页、活动详情展示。
2. **身份认证与授权（FR-02）**：图形验证码登录、邮箱验证码注册、JWT 会话管理、角色鉴权。
3. **活动发布与审核（FR-03）**：草稿创建与编辑、提交审核、审核通过/驳回、状态流转。
4. **个人参与管理（FR-04）**：报名、取消报名、收藏、取消收藏、个人日历、ICS 导出。
5. **发布者申请管理（FR-05）**：提交申请、管理员审批、角色变更。
6. **知识关联与订阅通知（FR-06）**：知识节点关联、订阅管理、通知生成与已读标记。
7. **数据源与内容质量（FR-07）**：数据源维护、抓取日志、质量分、疑似重复标识、URL 安全校验。
8. **审计与安全治理（FR-08）**：审计日志记录与查询、密码哈希存储、敏感日志脱敏。

2.2 非功能范围

1. **性能**：读接口 P95 响应时间不超过 500ms，写接口不超过 1s（本地/CI 环境基准）。
2. **易用性**：页面在 360px 以上视口无横向溢出，表单校验在输入位置即时提示。
3. **安全性**：越权请求返回 403，XSS 注入被转义，外部 URL 抓取过滤内网地址。

2.3 排除范围

以下内容不纳入本次系统测试范围：第三方邮件服务的送达率和延迟、外部搜索引擎的索引质量和排序算法、AI 服务的生成内容准确性，以及数据库在高并发下的性能压力测试。这些依赖以 Mock/降级合同验证为主，不以真实第三方服务的稳定性评价本系统的正确性。

3 测试策略

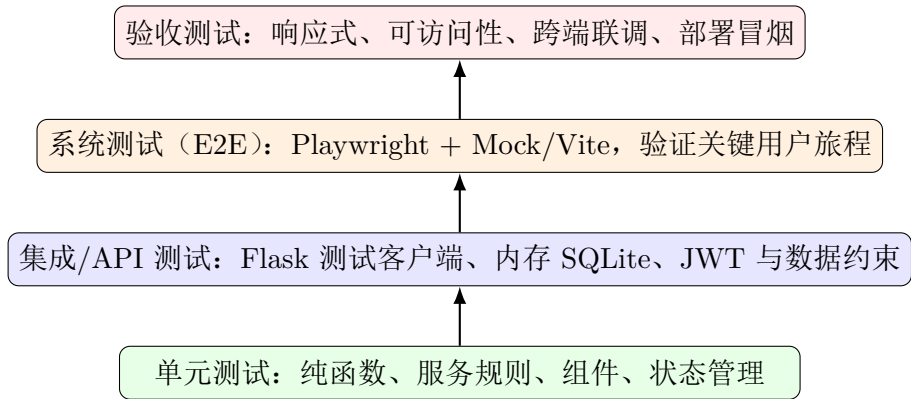


图 1：测试金字塔——从单元到验收逐层覆盖

3.1 单元测试

前端使用 Vitest + @vue/test-utils 对以下模块执行单元测试：认证 Store 的登录/登出状态管理、活动排序工具函数、活动质量评分函数、日历卡片组件渲染、活动详情组件状态分支。后端使用 pytest 对服务层纯函数执行单元测试，包括内容质量评分、去重判定、安全 URL 校验、知识节点匹配和搜索日志格式化。单元测试不依赖数据库、网络 and 外部服务。

3.2 集成测试

后端集成测试使用 Flask 测试客户端和内存 SQLite 数据库，conftest.py 提供隔离的测试应用实例、测试 JWT、以及典型用户、活动和数据源夹具。覆盖的 API 模块包括：auth（登录、注册、用户信息）、posters（活动 CRUD、状态转换）、audit logs（审计记录写入和查询）、data sources（创建、校验、列表）、knowledge（节点管理、关联）、search（内部搜索、外部搜索降级）、quality（质量评分）、dedup（疑似重复检测）、security（URL 校验、HTML 转义）。

3.3 系统测试

系统测试（端到端测试）使用 Playwright 驱动 Chromium 浏览器，在 Mock 服务模式下验证完整用户旅程：访客浏览活动列表和详情页、用户注册和登录流程、发布者创建

活动和提交审核、管理员审核活动和审批申请。系统测试覆盖页面跳转、权限导航、加载态和错误态等用户可见行为。

3.4 验收测试

验收测试以手工方式执行，覆盖三类场景：

- 1. **响应式布局：**在 375px、768px、1024px 和桌面宽度下检查页面布局，确认无水平溢出、按钮遮挡或关键信息丢失。
- 2. **可访问性：**通过 Tab 键遍历页面主要交互元素，确认焦点可见、操作可触达。
- 3. **部署冒烟：**使用 Docker Compose 启动完整服务后，访问健康检查接口、执行登录和活动查询等最小业务流程。

第三方邮件、外部搜索和 AI 服务以 Mock 验证为主，不以真实第三方稳定性评价系统正确性。

4 测试过程

4.1 测试环境

项目	规定
前端环境	Node 22 (CI) / Node 18+ (本地)；npm ci 安装依赖；Vite 开发服务器；Mock 服务 (node mock/index.js)；Chromium 浏览器 (Playwright)。
后端环境	Python 3.10+ 虚拟环境；依赖从 requirements-dev.txt 安装；测试使用 sqlite:///memory: 内存数据库；禁用真实 Redis、邮件服务和 LLM 调用。
CI 环境	GitHub Actions Ubuntu 最新版；actions/setup-node@v4 配置 Node 22；npx playwright install --with-deps chromium 安装浏览器。
准入条件	相关需求编号和接口契约已确认；测试数据和测试账号可用；项目构建成功；外部依赖可 Mock 或可降级。
退出条件	P0/P1 缺陷为零；受影响自动化测试全部通过；关键手工验收用例通过；遗留风险已记录并指定负责人。

4.2 测试计划

测试活动在项目开发周期内按迭代分阶段执行：

- 1. 第一阶段（第 1–3 周）：搭建测试基础设施，配置 Vitest、Playwright 和 pytest；编写认证模块（FR-02）的 API 集成测试和前端组件单元测试。
- 2. 第二阶段（第 4–7 周）：覆盖活动发布与审核（FR-03）和个人参与（FR-04）的测试用例；添加端到端测试覆盖访客浏览和登录注册流程。
- 3. 第三阶段（第 8–10 周）：补充审计（FR-08）、知识通知（FR-06）和数据治理（FR-07）的测试覆盖；执行兼容性和响应式验收测试。
- 4. 第四阶段（第 11–12 周）：全量回归测试；缺陷修复验证；测试报告和覆盖率数据归档。

4.3 测试用例设计

4.3.1 功能测试用例

编号	需求/场景	步骤与输入	预期结果
TC-01	认证成功/失败（FR-02）	使用正确凭据登录；使用错误密码登录；缺少必填字段登录。	成功返回 token 和用户信息；错误密码返回 401；缺字段返回 400。
TC-02	未认证与越权控制（FR-02）	无 token 访问 /auth/me；viewer 角色用户调用数据源管理接口。	无 token 返回 401；越权返回 403；不产生写入副作用。
TC-03	发布审核状态流转（FR-03）	publisher 创建 draft 草稿 → 提交审核；admin 批准/驳回；重复提交。	状态按 draft→pending_review→published/rejected 迁移；非法迁移返回 400；审核意见和审计日志存在。
TC-04	报名/日历幂等（FR-04）	同一用户对同一活动重复报名；重复加入日历；取消报名。	仅存在一条关联记录；取消后报名和日历事件同步移除。
TC-05	数据源校验（FR-07）	缺名称/URL 创建数据源；viewer 创建数据源；ftp 协议 URL。	参数错误 400；越权 403；拒绝非 http/https 协议和内网地址。
TC-06	内容安全（FR-07）	以 <script> 标签作为活动标题提交；抓取结果含控制字符。	输出已 HTML 转义；不执行脚本；控制字符被过滤。

编号	需求/场景	步骤与输入	预期结果
TC-07	检索与降级 (FR-01)	内部搜索空关键词、存在结果；外部搜索服务不可达。	内部返回稳定字段和排序；外部搜索降级返回 query/results/count/source/error 结构。
TC-08	前端用户旅程	访客浏览活动列表 → 进入详情；登录后创建活动 → 提交审核；管理员审核。	URL、视图、权限导航和错误态符合设计；E2E 可重复执行。
TC-09	发布者申请 (FR-05)	普通用户提交发布者申请；管理员审批通过/驳回。	申请记录创建成功；审批后用户角色变更。
TC-10	知识订阅通知 (FR-06)	用户订阅知识节点；管理员发布关联该节点的活动。	匹配活动发布后生成站内通知；用户可查看并标记已读。

4.3.2 性能测试用例

编号	场景	测试方法	预期指标
TC-P01	活动列表响应时间	使用 API 测试工具连续请求活动列表 50 次，统计 P95 延迟。	P95 < 500ms（本地/CI 环境）。
TC-P02	登录接口响应时间	连续执行登录请求 30 次（含验证码校验），统计 P95 延迟。	P95 < 1s（含验证码发送）。
TC-P03	前端首页加载	使用 Playwright 测量首页首屏加载时间。	模拟校园网环境下 < 2s。
TC-P04	并发请求稳定性	对活动列表接口发起 50 并发请求。	错误率 < 1%。

4.3.3 兼容性测试用例

编号	场景	测试方法	预期结果
TC-C01	视口宽度兼容	在 375px、768px、1024px、1920px 宽度下检查主要页面。	无水平溢出、无按钮遮挡、关键信息完整可见。
TC-C02	浏览器兼容	在 Chromium（Playwright）中执行端到端测试。	全部 E2E 测试通过。
TC-C03	键盘导航	使用 Tab 键遍历活动详情页、登录表单和管理页面。	焦点顺序合理，主要交互元素可键盘触达。

4.3.4 异常与边界测试用例

编号	场景	测试方法	预期结果
TC-E01	网络异常	前端在网络断开状态下操作。	页面显示网络错误提示，不产生后台请求。
TC-E02	空数据状态	无已发布活动时访问活动列表；无通知时访问通知页面。	显示空态提示和引导（如”暂无活动”）。
TC-E03	非法输入	超长标题（>200 字符）；负数的活动 ID；非法日期格式。	服务端返回 400 和明确错误信息；前端显示校验提示。
TC-E04	SSRF 防护	提交 file:// 协议、10.0.0.1 内网地址、localhost 的抓取目标 URL。	请求被拒绝，返回安全校验错误。
TC-E05	并发重复提交	对同一活动快速连续发送两次报名请求。	数据库唯一约束保证仅一条报名记录。

4.4 测试执行结果

本次复核执行记录如下（2026-07-18）：

检查项	实际命令/观察	结论与后续动作
前端聚合验证	<code>npm run verify</code> 输出至 <code>vue-tsc -b && vite build</code> 阶段, 后续 <code>unit/E2E</code> 结果未出现	未完成。需在开发机或 CI 采集完整日志。
前端单独构建	<code>npm run build</code> 同样仅输出至 <code>vue-tsc -b && vite build</code>	构建阻塞待确认, 需定位类型检查阻塞点。
后端 pytest	<code>pytest -q -p no:cacheprovider</code> 返回 <code>No module named pytest</code>	环境缺陷, 需安装开发依赖后重跑。
LaTeX 编译	六份报告 XeLaTeX 编译通过, 图表经抽样检查	文档可正常生成。

5 缺陷管理

5.1 缺陷分类与严重度

缺陷按严重度从高到低分为四个级别, 每级附带具体示例和处置时限:

- P0 (致命):** 安全泄露 (如 JWT 密钥或数据库密码提交到仓库)、数据破坏 (如审核通过后活动内容丢失)、全站不可用 (如 API 启动崩溃、数据库连接失败)。**处置:** 立即停止发布并回滚版本, 24 小时内修复, 修复后补充安全审查记录。
- P1 (严重):** 核心业务流程受阻且无可替代方案。例如: 登录接口返回 500 导致所有用户无法登录、活动提交审核后状态未更新 (发布者无法确认审核进度)、报名入口报错 (用户无法参与活动)。**处置:** 当前迭代内修复, 必要时中断非核心功能开发。
- P2 (一般):** 非核心功能缺陷, 存在可接受的替代方案。例如: 搜索结果的排序不够精确、通知未实时推送但在下次登录时可同步、管理后台的筛选条件重置。**处置:** 可安排至下一迭代修复。
- P3 (建议):** 界面样式偏差、文案错误、文档过期。例如: 按钮颜色与设计稿不一致、提示文案有错别字、API 文档中的示例字段与实际返回不匹配。**处置:** 视资源情况安排修复。

缺陷按发现阶段分为编码阶段自测发现、代码审查发现、单元测试发现、集成测试发现、端到端测试发现和手工验收发现。越早发现缺陷修复成本越低, 因此编码阶段的类型检查和代码审查是性价比最高的质量活动。

5.2 缺陷生命周期与字段

缺陷从发现到关闭经历以下状态流转:

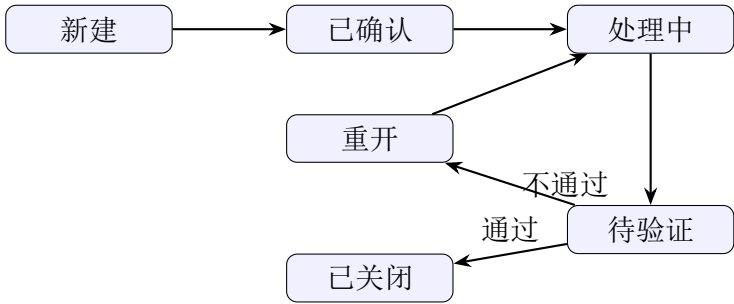


图 2：缺陷状态流转

每个缺陷单记录以下完整字段：

- 1. **基本信息：**缺陷编号（格式 QA-xxx）、发现版本号、发现环境（开发/测试/预发布）、发现阶段（编码/审查/单测/集成/E2E/验收）、报告人和报告日期。
- 2. **关联追踪：**关联需求编号（FR-xx）或关联测试用例编号（TC-xx），用于需求—测试—缺陷的双向追溯。例如缺陷”登录验证码过期时间不符”关联 FR-02 和 TC-01。
- 3. **复现信息：**复现步骤（Given-When-Then 格式）、实际结果、期望结果、日志片段或截图附件。
- 4. **分级与指派：**严重度（P0-P3）、优先级（高/中/低）、负责人、目标修复版本。
- 5. **验证与关闭：**修复验证人、验证日期、关联的回归测试名称或 ID。已关闭缺陷必须关联回归测试用例，或明确说明不适合自动化验证的原因。

6 质量保证方法

6.1 代码审查

每个合并请求至少由一名非作者成员审查，使用以下逐项检查清单：

- 1. **权限安全：**受限接口是否在服务端通过 JWT + 角色装饰器校验？前端是否根据角色隐藏或禁用越权入口？新增接口是否默认鉴权而非默认开放？
- 2. **异常路径：**代码是否覆盖以下异常场景——空数据（列表/详情返回空态）、网络失败（请求超时或断开显示错误提示）、参数非法（后端返回 400 和明确错误信息）、越权访问（返回 403 且不产生写入副作用）？
- 3. **代码质量：**命名是否清晰表达意图？函数是否超过 50 行需要拆分？条件逻辑嵌套是否超过三层需要提前返回？注释是否解释”为什么”而非”是什么”？
- 4. **测试覆盖：**新增功能是否有对应的单元测试？涉及跨端流程是否有端到端测试？测试是否覆盖正向路径和至少一个异常路径？是否有被 ‘skip’ 跳过但未说明原因的测试用例？

5. **配置安全**：是否引入了新的敏感信息（API Key、密码）？是否有硬编码的环境相关值（数据库地址、域名）？`.env.example` 是否同步更新？

审查结果记录在合并请求评论中，所有检查项确认通过后方可合并。CI 失败、含真实密钥或破坏主干构建的合并请求禁止合并。紧急缺陷修复可走快速通道合并，但必须在修复后 24 小时内补齐测试用例和审查记录。

6.2 静态检查

前端在构建时通过 `vue-tsc -b` 执行 TypeScript 严格模式类型检查（`tsconfig.json` 中 `strict: true`），自动捕获以下问题：

1. 接口字段不匹配：前端 API 响应类型定义与组件期望的属性类型不一致时编译报错。
2. 空值引用：变量可能为 `null` 或 `undefined` 时未经判空直接访问其属性。
3. 类型错误：函数参数类型不符、枚举值超出定义范围、JSON 解析结果未指定类型。

后端计划集成 `pylint` 或 `flake8` 在 CI 中执行静态检查，当前阶段以人工审查补充。静态检查门禁的目标是在编码阶段捕获 80% 以上的类型和接口相关问题，降低运行时缺陷。

6.3 自动化测试

三项自动化测试手段已在项目中落地，通过 `npm run verify` 聚合执行：

1. **Vitest 单元测试（5 个测试文件）**：覆盖认证 Store 的登录/登出状态变化、活动排序函数的多条件排序逻辑、活动质量评分函数的评分计算规则、日历卡片组件的渲染和事件绑定、活动详情组件的加载/就绪/错误三种状态分支。
2. **Playwright 端到端测试（1 个测试文件）**：使用 Chromium 浏览器在 Mock 服务模式验证访客浏览活动列表、用户注册和登录流程、受保护页面的未登录跳转。测试用例覆盖页面跳转、权限导航、表单校验提示和错误态展示。
3. **pytest 集成测试（20+ 测试文件）**：使用 Flask 测试客户端和内存 SQLite，覆盖 `auth`（登录/注册/用户信息）、`posters`（活动 CRUD/状态流转）、`audit`（日志写入和查询）、`data sources`（创建/校验/列表）、`knowledge`（节点管理/关联）、`search`（内部搜索/外部降级）、`quality`（质量评分）、`dedup`（疑似重复检测）、`security`（URL 校验/HTML 转义）等模块。

6.4 回归测试

自动化回归测试在以下三种场景中自动触发：

1. **每次 push**：CI 执行全部单元测试和端到端测试，确保提交不破坏已有功能。
2. **Pull request 创建/更新**：在 push 触发的基础上，合并前必须全部通过。

3. **缺陷修复验证**：关联缺陷的回归测试用例在修复提交时自动执行，确认修复有效且不引入新问题。

端到端测试覆盖访客浏览、登录注册和活动搜索等关键用户旅程，确保功能迭代不破坏已有正确行为。回归测试的通过率为合并请求的硬性门禁——未全部通过不得合并。

6.5 发布检查

版本发布前的质量门禁按以下检查清单逐项确认：

检查项	确认方式	门禁标准
CI 工作流	GitHub Actions 运行结果	全部通过（构建 + 单测 + E2E）
缺陷状态	缺陷跟踪记录确认	目标版本 P0/P1 为零
接口契约	契约矩阵与实际 API 对比	接口变更后已更新矩阵文档
部署冒烟	Docker Compose 启动 + 健康检查	/api/health 返回 200
版本标识	Git tag + release notes	版本 tag 已创建，release notes 已编写

所有检查项确认通过后，由负责人在 release notes 中签署发布确认。任一检查项未达标则暂停发布，修复达标后方可继续。

7 测试结论

通过上述测试策略和质量保证方法，逸仙活动云项目建立了覆盖单元、集成、系统和验收四个层次的测试体系。前端自动化测试（Vitest + Playwright）已集成到 CI 流水线，后端 pytest 测试资产覆盖全部核心模块。当前阶段的主要限制是后端 CI 尚未启用、前端构建存在阻塞待确认、以及性能基线和无障碍扫描等属于后续补充项。

测试与需求的可追溯性方面：FR-02 对应认证 API 测试、auth store 测试和 test_auth_api.py；FR-03 对应活动编辑/审核页、posters API 测试和审计测试；FR-05/FR-07 对应知识、去重、质量、搜索和安全服务测试；FR-08 对应 Axios 拦截器、页面状态和 Playwright 授权流程。每新增需求应至少补充一个正向和一个异常/权限测试用例；修复缺陷应关联回归测试用例。

在完成 QA-001 和 QA-002 的环境修复、以及后端 CI 和契约自动化测试的补充后，本系统的测试体系可达到可提交、可回归、可发布的质量门禁标准。